《算法分析与设计》

实验报告

学　　号 2024112639

姓　　名 毛露涵

年　　级 2024 级

专　　业 软件工程

院　　系 计算机与人工智能学院

实验预习（40%）	上机实验（40%）	文档排版（20%）	总分	批阅人签字

2025-2026 学年第 2 学期

实验 8.3 机器人搬运货物

1. 实验目的

（1）理解分支限界法的求解过程。

（2）分析分支限界法的时间复杂度，比较分支限界法算法与其他算法的时间效率差异。

（3）学会如何利用分支限界法求解具体问题，了解动分支限界法的应用范围及在实际应用中的局限性。

2. 实验任务

（1）完成实验 8.3（机器人搬运货物问题）的各项要求。

（2）用 C++语言实现该算法并进行测试。

（3）撰写实验报告，实验报告内容包括实验目的、实验任务、实验环境、实验步骤、实验结果和实验总结等部分。

3. 实验环境

3.1 硬件环境

（1） 计算机：Legion R9000P ADR10H

（2） CPU: AMD Ryzen 9 8945HX with Radeon Graphics

（3） RAM：32GB

3.2 软件环境

（1） 操作系统：Windows11 家庭中文版

（2） 开发工具：Visual Studio 2026

4. 实验步骤及结果

4.1 实验预习

4.1.1 给出采用分支限界法求解该问题时的目标函数，约束条件以及限界函数

本实验中的仓库由 n × m 个方格组成，方格中 # 表示墙体，. 表示空白方格，S 表示机器人初始位置，B 表示货物初始位置，T 表示货物目标位置。机器人必须先移动到货物相邻的空白方格，然后推动货物移动。每推动货物从一个方格到另一个方格，记为货物移动一次。

设一个搜索结点为：

Node = (bx, by, rx, ry, step)

其中：

(bx, by)：货物当前位置

(rx, ry)：机器人当前位置

step：货物已经被推动的次数

目标函数：min step

即在货物到达目标位置 T 的所有方案中，使货物移动次数最少。

约束条件：

（1）机器人和货物不能移动到墙体 #。

（2）机器人和货物不能移动到仓库边界外。

（3）机器人不能穿过货物所在方格。

（4）机器人只有能够到达货物反方向相邻格时，才可以推动货物。

（5）推动货物后，货物的新位置必须是可通行空格或目标位置。

限界函数：

设当前货物位置为 (bx, by)，目标位置为 (tx, ty)，当前已经推动次数为 step，采用曼哈顿距离作为估计代价：

h = |bx - tx| + |by - ty|

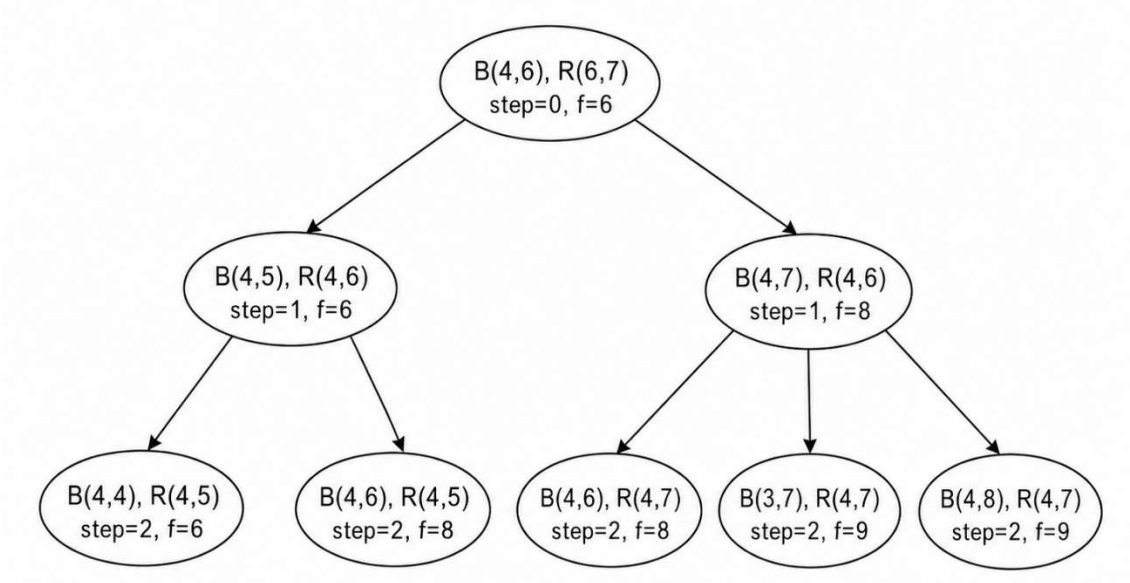
则限界函数为：

f = step + h

其中，step 是当前已经付出的代价，h 是货物到目标位置的理论最少推动次数估计。分支限界法使用小根堆优先扩展 f 值最小的结点。

4.1.2 给出采用分支限界法求解样例输入时的解空间树

结点格式：B（货物位置），R（机器人位置），step=推动次数，f=限界函数值



4.1.3 分析采用分支限界法求解样例输入时的求解过程，堆结点的定义，堆结点的值以及堆中元素的变化过程

堆结点定义如下：Node = (bx, by, rx, ry, step, h, f)

其中：

bx, by：货物当前位置

rx, ry：机器人当前位置

step：当前货物已经移动的次数

h：货物当前位置到目标位置的曼哈顿距离

f：限界函数值，f = step + h

小根堆按照 f 值从小到大排列。如果 f 值相同,则优先扩展 step 较小的结点。

样例输入下，主要求解过程如下：

初始结点：

B(4,6), R(6,7), step=0

h=|4-2|+|6-2|=6

f=0+6=6

堆中元素变化过程如下：

初始：

[(B(4,6), R(6,7), step=0, h=6, f=6)]

扩展 B(4,6) 后：

[(B(4,5), R(4,6), step=1, h=5, f=6),
(B(4,7), R(4,6), step=1, h=7, f=8)]

扩展 B(4,5) 后：

[(B(4,4), R(4,5), step=2, h=4, f=6),
(B(4,7), R(4,6), step=1, h=7, f=8)]

扩展 B(4,4) 后：

[(B(4,3), R(4,4), step=3, h=3, f=6),
其他 f=8 的结点]

扩展 B(4,3) 后：

[(B(4,2), R(4,3), step=4, h=2, f=6),
其他 f=8 的结点]

扩展 B(4,2) 后：

[(B(3,2), R(4,2), step=5, h=1, f=6),
其他 f=8 的结点]

扩展 B(3,2) 后：

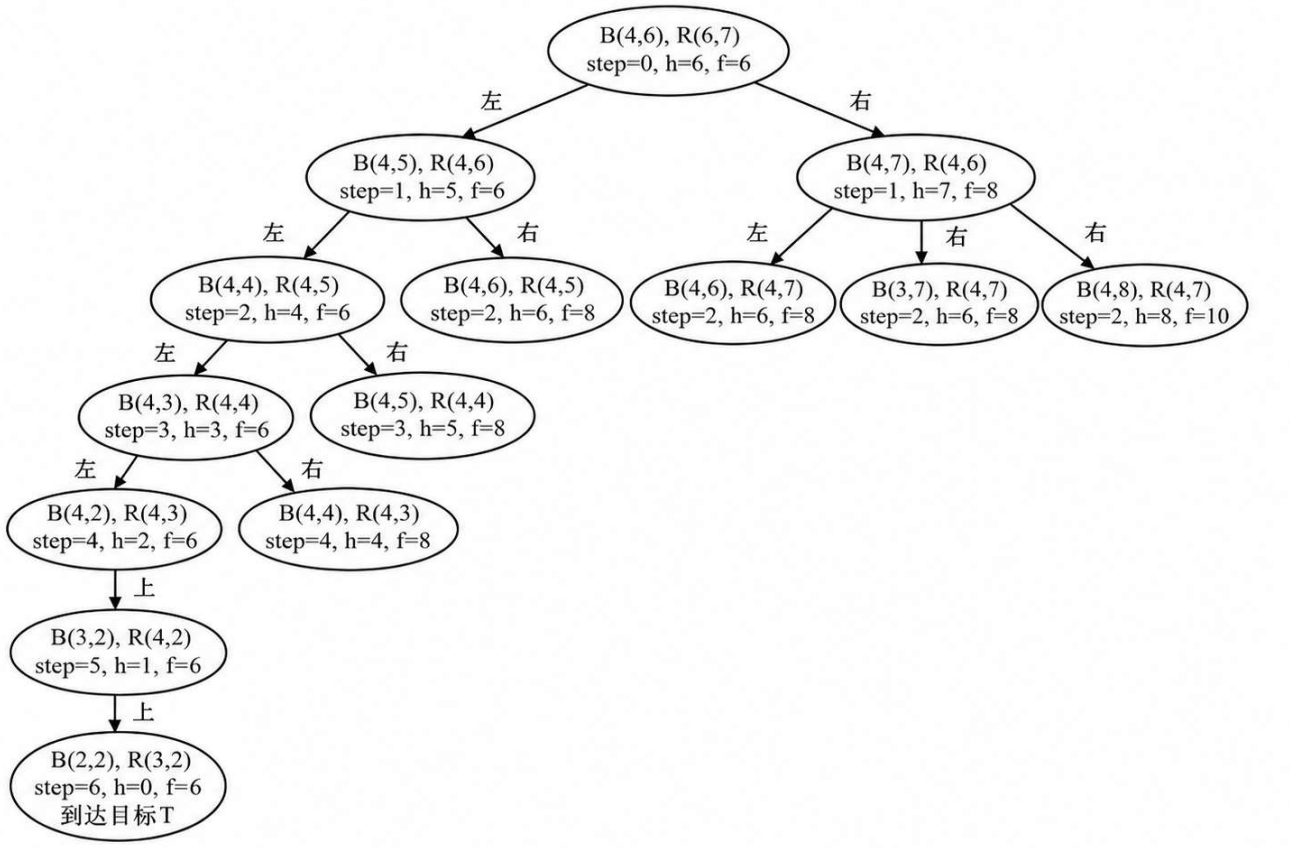
[(B(2,2), R(3,2), step=6, h=0, f=6),
其他 f=8 的结点]

扩展 B(2,2) 时，货物已经到达目标位置，算法结束。

因此样例输出为：6

4.1.4 画出采用分支限界法求解样例输入时的搜索空间树

结点格式：B（货物位置），R（机器人位置），step=推动次数，h=曼哈顿距离，f=step+h



4.1.5 用 C/C++语言编写程序求解上述问题

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <queue>
#include <vector>
#include <map>
#include <cmath>
using namespace std;

int n, m;
```

```
vector<string> grid;
int sx, sy, bx, by, tx, ty;

int dx[4] = { -1, 1, 0, 0 };
int dy[4] = { 0, 0, -1, 1 };

struct State {
    int bx, by, rx, ry;

    bool operator < (const State& o) const {
        if (bx != o.bx) return bx < o.bx;
        if (by != o.by) return by < o.by;
        if (rx != o.rx) return rx < o.rx;
        return ry < o.ry;
    }
};

struct Node {
    int bx, by;
    int rx, ry;
    int step;
    int h;
    int f;
};

struct Cmp {
    bool operator()(const Node& a, const Node& b) const {
        if (a.f != b.f) return a.f > b.f;
        return a.step > b.step;
    }
};

bool freep(int x, int y) {
    if (x < 0 || x >= n || y < 0 || y >= m) return false;
```

```
    return grid[x][y] != '#';
}

int geth(int x, int y) {
    return abs(x - tx) + abs(y - ty);
}

bool reach(int rx, int ry, int gx, int gy, int cbx, int cby) {
    if (!freep(gx, gy)) return false;
    if (gx == cbx && gy == cby) return false;

    queue<pair<int, int> > q;
    vector<vector<int> > vis(n, vector<int>(m, 0));

    q.push(make_pair(rx, ry));
    vis[rx][ry] = 1;

    while (!q.empty()) {
        pair<int, int> cur = q.front();
        q.pop();

        if (cur.first == gx && cur.second == gy) {
            return true;
        }

        for (int i = 0; i < 4; i++) {
            int nx = cur.first + dx[i];
            int ny = cur.second + dy[i];

            if (!freep(nx, ny)) continue;
            if (nx == cbx && ny == cby) continue;
            if (vis[nx][ny]) continue;

            vis[nx][ny] = 1;
```

```
            q.push(make_pair(nx, ny));
        }
    }

    return false;
}

int solve() {
    priority_queue<Node, vector<Node>, Cmp> heap;
    map<State, int> dist;

    Node st;
    st.bx = bx;
    st.by = by;
    st.rx = sx;
    st.ry = sy;
    st.step = 0;
    st.h = geth(bx, by);
    st.f = st.step + st.h;

    heap.push(st);

    State ss;
    ss.bx = bx;
    ss.by = by;
    ss.rx = sx;
    ss.ry = sy;
    dist[ss] = 0;

    while (!heap.empty()) {
        Node cur = heap.top();
        heap.pop();

        State cs;
```

```
cs.bx = cur.bx;
cs.by = cur.by;
cs.rx = cur.rx;
cs.ry = cur.ry;

if (!dist.count(cs) || dist[cs] != cur.step) continue;

if (cur.bx == tx && cur.by == ty) {
    return cur.step;
}

for (int i = 0; i < 4; i++) {
    int needx = cur.bx - dx[i];
    int needy = cur.by - dy[i];
    int nbx = cur.bx + dx[i];
    int nby = cur.by + dy[i];

    if (!freep(nbx, nby)) continue;

    if (!reach(cur.rx, cur.ry, needx, needy, cur.bx, cur.by)) {
        continue;
    }

    Node nxt;
    nxt.bx = nbx;
    nxt.by = nby;
    nxt.rx = cur.bx;
    nxt.ry = cur.by;
    nxt.step = cur.step + 1;
    nxt.h = geth(nbx, nby);
    nxt.f = nxt.step + nxt.h;

    State ns;
    ns.bx = nxt.bx;
```

```
            ns.by = nxt.by;
            ns.rx = nxt.rx;
            ns.ry = nxt.ry;

            if (!dist.count(ns) || nxt.step < dist[ns]) {
                dist[ns] = nxt.step;
                heap.push(nxt);
            }
        }
    }

    return -1;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    while (cin >> n >> m) {
        grid.assign(n, "");

        sx = sy = bx = by = tx = ty = -1;

        for (int i = 0; i < n; i++) {
            cin >> grid[i];

            for (int j = 0; j < m; j++) {
                if (grid[i][j] == 'S') {
                    sx = i;
                    sy = j;
                }
                else if (grid[i][j] == 'B') {
                    bx = i;
                    by = j;
```

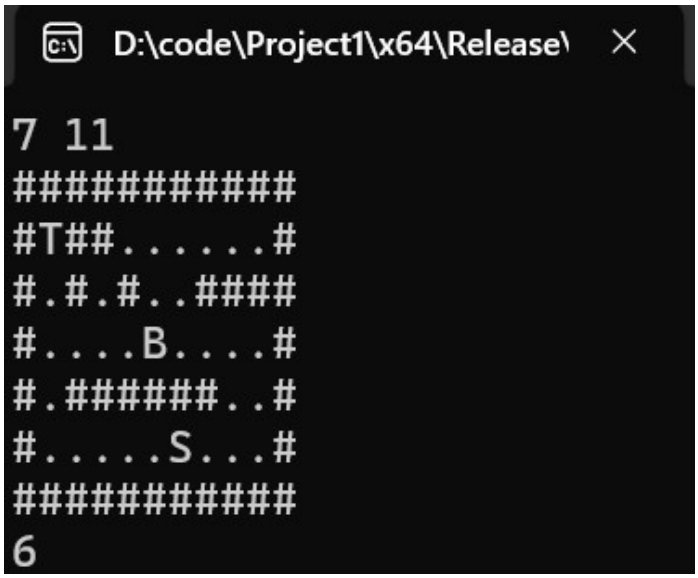
```
                }
                else if (grid[i][j] == 'T') {
                    tx = i;
                    ty = j;
                }
            }
        }

        cout << solve() << endl;
    }

    return 0;
}
```

4.2 上机实验

4.2.1 调试步骤（5）的程序，验证样例输入时程序执行过程中堆的变化过程是否与步骤（3）的分析结果一致



名称	值	类型
cur.bx + 1	4	int
cur.by + 1	6	int
cur.rx + 1	6	int
cur.ry + 1	7	int
cur.step	0	int
cur.h	6	int
cur.f	6	int

名称	值	类型
cur.bx + 1	4	int
cur.by + 1	5	int
cur.rx + 1	4	int
cur.ry + 1	6	int
cur.step	1	int
cur.h	5	int
cur.f	6	int

名称	值	类型
cur.bx + 1	4	int
cur.by + 1	4	int
cur.rx + 1	4	int
cur.ry + 1	5	int
cur.step	2	int
cur.h	4	int
cur.f	6	int

名称	值	类型
cur.bx + 1	4	int
cur.by + 1	3	int
cur.rx + 1	4	int
cur.ry + 1	4	int
cur.step	3	int
cur.h	3	int
cur.f	6	int

名称	值	类型
cur.bx + 1	4	int
cur.by + 1	2	int
cur.rx + 1	4	int
cur.ry + 1	3	int
cur.step	4	int
cur.h	2	int
cur.f	6	int

名称	值	类型
cur.bx + 1	3	int
cur.by + 1	2	int
cur.rx + 1	4	int
cur.ry + 1	2	int
cur.step	5	int
cur.h	1	int
cur.f	6	int

名称	值	类型
cur.bx + 1	2	int
cur.by + 1	2	int
cur.rx + 1	3	int
cur.ry + 1	2	int
cur.step	6	int
cur.h	0	int
cur.f	6	int

4.2.2 设计新的测试用例，对程序进行进一步验证

新的测试用例：

5 5

#####

#T..#

#B..#

#S..#

#####

手工计算过程如下：

T = (2,2)

B = (3,2)

S = (4,2)

机器人初始位置在货物下方，可以直接向上推动货物一次：

B(3,2) -> B(2,2)

此时货物到达目标位置，因此货物最少移动次数为：1



程序运行输出为：1

程序输出结果与手工计算结果一致，说明程序能够正确求解该测试用例。

5. 实验总结

本实验使用分支限界法求解机器人搬运货物问题。实验中将货物位置、机器人位置和货物推动次数作为搜索状态，并使用优先队列维护活结点。每次从堆中取出

限界函数值最小的结点进行扩展，从而优先搜索更可能得到最优解的路径。

在算法实现过程中，货物的每一次推动作为一层搜索，机器人是否能够到达货物反方向相邻位置则通过 BFS 进行判断。限界函数采用当前推动次数加货物到目标位置的曼哈顿距离，能够对搜索过程起到一定的引导作用。样例测试结果为 6，新测试用例结果为 1，均与手工分析一致。

影响本算法时间效率的主要因素包括仓库规模、墙体分布、机器人可达性判断次数、堆中活结点数量以及重复状态的数量。通过记录已访问状态、使用优先队列和限界函数，可以减少无效搜索，提高求解效率。但如果地图规模较大或障碍物结构复杂，算法仍然需要多次 BFS 判断机器人可达性，因此运行时间会明显增加。总体来说，分支限界法适合求解此类最优化路径问题。